Adapting Small Query Runtime Models to New Machines and Unseen Query Templates

Krish Patel
Email: kmp.patel@uwaterloo.ca
CS 680: Introduction to Machine Learning, University of Waterloo

Abstract

Query runtime models are trained on one machine and drift when the hardware changes. For small models that predict PostgreSQL runtime from 19 plan features, I ask how many queries measured on a new machine suffice to adapt them (Q1), and whether a model anchored to the optimizer's cost predicts runtime for templates it never saw (Q2). I timed 1,162 TPC-H and TPC-DS queries in six environments with identical plans. The slowdown varies mostly between templates, so a speed factor from five measured queries removes much of the error and then stops improving. A ridge regression of the old model's errors keeps improving on TPC-H: in Q1's primary test its error at 25 queries is 0.93 times the speed factor's (95% interval 0.89 to 0.96). A per-template correction from one measured query per template does better on both benchmarks (error 1.10 on TPC-H, 1.19 on TPC-DS), close to the same correction on a model trained on the new machine. On unseen templates nothing reliably beats the speed factor, and Q2's primary test fails (ratio to the cost line 1.04, interval 0.90 to 1.19). Code and data: github.com/KrishPatel13/query-runtime-transfer.

1 Introduction

A database system that knows how long a query will take can route short queries past long ones, hold back a query that would take hours, and size a cluster for a workload. PostgreSQL's optimizer estimates a cost for every plan, but in units whose relation to time depends on the machine and the plan (Wu et al. 2013). Learned models close this gap by mapping plan features to measured runtime (Ganapathi et al. 2009; Akdere et al. 2012; Marcus and Papaemmanouil 2019), and production systems use such predictions for scheduling (Wu et al. 2024).

A learned runtime model is trained on one machine. Move the database to other hardware, or give it fewer processors, and the same plans change speed by different factors (Figure 1a). In machine-learning terms the inputs do not change: the same queries get the same plans and features everywhere, so the input distribution is fixed and only the distribution of runtime given the features shifts. Reweighting unlabeled inputs, the usual answer to covariate shift (Shimodaira 2000), has nothing to correct; the model needs runtimes measured on the new machine, and each costs time there. The first question (Q1) is one of few-shot adaptation: how few queries measured on a target machine are enough to adapt a model trained on a source machine, which queries should they be, and does it keep improving as more arrive? The second (Q2) asks whether a model anchored to the optimizer's cost predicts runtime for templates never seen in training, which earlier work found hard for plan-level models (Akdere et al. 2012); each template acts as a domain. The contributions are:

- a dataset of 1,162 TPC-H and TPC-DS queries timed in three passes in six environments with identical plans (I found no public dataset that times the same plans on several machines), and an analysis of how their runtimes shift;

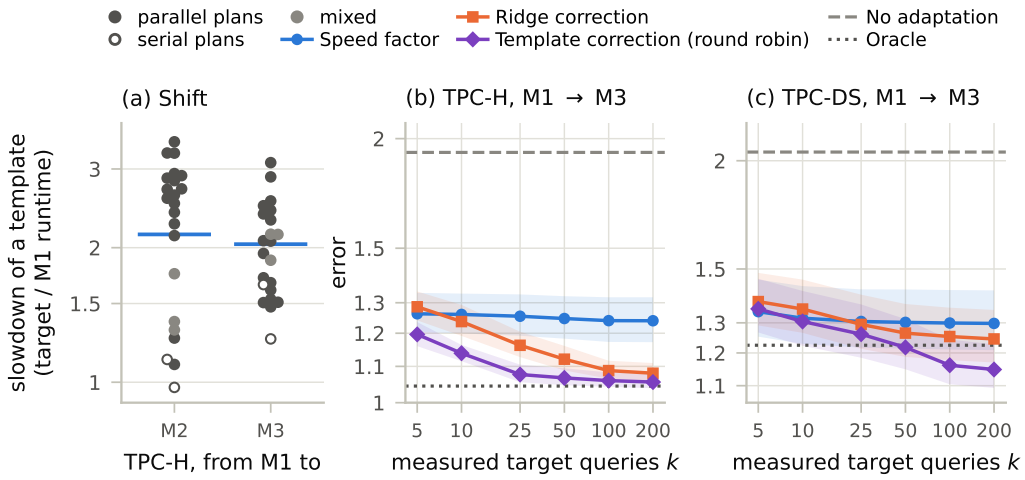


Figure 1: (a) Slowdown of each TPC-H template from M1 to M2 (one processor) and to M3 (the server): filled if all its plans have parallel workers, hollow if none, gray if mixed; the blue line is the geometric mean, the factor a speed factor estimates. (b, c) Error (Section 3) against the number k of measured target queries from M1 to M3, with 95% intervals over templates.

- for Q1, the primary test, in which a ridge correction of the source model's errors beats one speed factor, and an extension in which per-template offsets from one query per template come close to a model trained on the target, confirmed on 18 pairs measured after five hypotheses were fixed;
- for Q2, a negative result: on templates without measured queries no correction reliably beats the speed factor, and the cost-anchored model is no better than a line on the optimizer's cost.

2 Related Work

Runtime models and new hardware. Plan-level models predict runtime from plan features (Ganapathi et al. 2009) but transfer poorly to a held-out template (Akdere et al. 2012); plan-structured networks (Marcus and Papaemmanouil 2019), tree models (Sun and Li 2019) and tree transformers (Zhao et al. 2022) train on thousands of queries, explicit features on far fewer (Yu and Luo 2023), and all inherit wrong row estimates (Leis et al. 2015). For a new machine, Wu et al. (2013) calibrate PostgreSQL's cost units with designed queries, Zhu and Larson (1994) fit a cost formula per query class from sampled queries, and ParamTree predicts cost constants from hardware and settings (Yang et al. 2023); zero-shot models transfer across databases on one machine (Hilprecht and Binnig 2022). QCFE retrains on 200 rerun queries (Yan et al. 2024), DACE fine-tunes on a whole rerun workload (Liang et al. 2024), T3 reruns its benchmark (Rieger and Neumann 2025), and Redshift feeds the instance type to a global model (Wu et al. 2024). These need many target queries or a description of the hardware; I ask how few measured queries suffice, and which.

Transferring and updating models. For configurable software, a line from source to target performance (model shift) moves a model across hardware with 5 to 15 runs (Valov et al. 2017; Jamshidi et al. 2017) and fails for severe changes (Valov et al. 2020; Jamshidi et al. 2018); OneProxy adapts a latency predictor with one global factor plus sparse per-operator deviations (Lu et al. 2021). Clinical prediction calls the speed factor *recalibration in the large* and finds it hard to beat unless the new sample is large (Steyerberg et al. 2004; Vergouwe et al. 2017). Kuzborskij and Orabona (2013) analyze least squares regularized toward a source hypothesis, which my ridge correction is; learning the residuals of an analytical model follows Didona et al. (2015); and learned database models have been updated for shifts in data and workload (Kurmanji and Triantafillou 2023; Wu and Ives 2024).

Choosing what to measure. Farthest-first selection covers a feature space (Gonzalez 1985; Sener and Savarese 2018), D-optimal design pins down a linear model (Wynn 1970; Mitchell 1974), and Ernest picks training runs by optimal design under a cost budget (Venkataraman et al. 2016). Streams

are scored test-then-train (Gama et al. 2013). I claim only prediction accuracy, since better predictions need not give better plans (Heinrich et al. 2025).

3 Problem Setup

For a query q from template $\tau(q)$ (one SQL text whose queries differ only in parameters), let $x(q) \in \mathbb{R}^{19}$ be the features of plain `EXPLAIN` (PostgreSQL Global Development Group 2026): estimated startup and total cost, rows and width; node count and depth; the largest and summed log row estimates over nodes; counts of scans, joins, sorts, aggregates, materializations and subplans by type; and planned parallel workers. Let $y_m(q)$ be its runtime in environment m. The plan, and so $x(q)$, is the same in every environment (Section 5). Models predict $\log y$; accuracy is the Q-error $Q(y, \hat{y}) = \max(y/\hat{y}, \hat{y}/y)$ (Moerkotte et al. 2009).

The shift. On a target t, a source model f_s trained on environment s has log error $e(q) = \log y_t(q) - f_s(x(q)) = \delta(q) + u_s(q)$: the log slowdown $\delta(q) = \log(y_t(q)/y_s(q))$ plus the model's residual $u_s(q)$ on the source. The corrections below assume δ is a constant (speed factor), a function of x (ridge correction) or a value per template (template offsets). The template share

$$\eta^2 = \textstyle\sum_\tau n_\tau(\bar{\delta}_\tau - \bar{\delta})^2 \big/ \sum_q(\delta(q) - \bar{\delta})^2, \tag{1}$$

with n_τ queries of mean slowdown $\bar{\delta}_\tau$ in template τ, is the part of the variance of δ between templates; computed between two passes on one machine, it measures noise.

Q1 and Q2. Queries are split once, inside every template, into a pool P and a test set T with templates $\mathcal{T}$, the same in every environment. A selection rule picks k pool queries without running them on the target; their target runtimes and f_s give the adapted model f_t, scored on T. Random selection follows one of $R = 20$ arrival orders shared by all methods. For Q2, models are trained without a template g (TPC-DS: its family of near copies) and scored on g.

Metrics. For a method, a k and an order r, each test template gets the median Q-error $\tilde{Q}_{\tau r}$ of its test queries; the error

$$E(k) = \exp\Big(\tfrac{1}{|\mathcal{T}|}\textstyle\sum_{\tau\in\mathcal{T}}\tfrac{1}{R}\sum_{r=1}^{R}\log\tilde{Q}_{\tau r}\Big)$$

is a geometric mean in which every test template counts once. Two methods are compared by the geometric mean over templates of the ratio of their errors, with a 95% interval from 2000 resamples of whole templates (TPC-DS: families) (Efron and Tibshirani 1993), paired across methods. With E_{none} for the unadapted model and E_{oracle} for the source pipeline retrained on the whole target pool, $G(k) = \log(E_{\text{none}}/E(k))/\log(E_{\text{none}}/E_{\text{oracle}})$ is the share of the gap closed. The oracle knows nothing of templates, so it is a reference, not a bound ($G > 1$ is possible).

4 Methods

Source models. The *cost line* regresses log runtime on $\log(1 + \kappa(x))$, with $\kappa(x)$ PostgreSQL's total cost; *ridge regression* (Hoerl and Kennard 1970) uses the standardized features; the *random forest* (Breiman 2001) has 300 trees, two samples per leaf and $\sqrt{19}$ features per split; and the *correction model*, fixed before the study as f_s, adds a random forest on the cost line's residuals, so that it falls back to the cost line where the forest learned little. The template and global means ignore the plan. All use scikit-learn (Pedregosa et al. 2011).

Corrections. With e_i the source model's log error on the k measured queries and $\bar{e}$ their mean, each correction predicts $\log\hat{y} = f_s(x) + c(x)$:

- *Speed factor*: $c = \bar{e}$, one factor on runtime.
- *Ridge correction*: $c(x) = \hat{\beta}_0 + \hat{\beta}^\top\tilde{x}$, with $\tilde{x}$ the features standardized over the k queries and

$$(\hat{\beta}_0, \hat{\beta}) = \arg\min_{\beta_0,\beta}\ \textstyle\sum_{i=1}^{k}\big(e_i - \beta_0 - \beta^\top\tilde{x}_i\big)^2 + \lambda\|\beta\|^2, \tag{2}$$

  the intercept unpenalized and λ chosen by leave-one-out (Golub et al. 1979) among 45 values from 10^{-6} to 10^5.
- *Affine+parallel*: $c(x) = a + b\,f_s(x) + \pi(x)(a' + b'\,f_s(x))$, with $\pi(x) = 1$ for plans with parallel workers, fitted like (2) with 2 to 4 parameters.

Algorithm 1: Adapting a source model with measured target queries and continuing to learn

Input: source model f_s; target pool P; selection rule; correction c; budget k; window W (all, or 50)

$S \leftarrow$ the first k queries chosen from P `// before any target run`

run each $q \in S$ on the target; $e_q \leftarrow \log y_t(q) - f_s(x(q))$

fit c to $\{(x(q), \tau(q), e_q) : q \in S\}$; $f_t \leftarrow f_s + c$ `// C2 ends here`

for *each later batch B of queries* **do**

  predict f_t on B, run B, add it to S `// test-then-train`

  refit c on the last W queries of S `// C3`

- *Template offsets* and the *template correction*: the speed factor, or affine+parallel, plus for each measured template the mean residual of its queries; other templates get the factor or line alone.
- *1-NN*: the error of the nearest measured query in the standardized feature space, without template labels.

As baselines, *target only* trains the source pipeline on the k queries, the *pooled refit* on the source pool plus the k queries with the environment as a feature, the *forest on the errors* fits a forest to the e_i, the *template lookup* predicts a template's mean measured log runtime, and the *oracle* trains on the whole target pool. The *no-move control* applies a correction to a model trained on the target pool itself, so the gap to it is the cost of the move.

Proposition 1. *For every $\lambda > 0$, the solution of* (2) *has $\hat{\beta}_0 = \bar{e}$ and $\hat{\beta} = (\tilde{X}^\top \tilde{X} + \lambda I)^{-1} \tilde{X}^\top (e - \bar{e}\mathbf{1})$, where $\tilde{X}$ stacks the $\tilde{x}_i$, and $\|\hat{\beta}\| \le \|\tilde{X}^\top (e - \bar{e}\mathbf{1})\|/\lambda$. Hence as $\lambda \to \infty$ the ridge correction becomes the speed factor.*

Proof. The standardized features sum to zero over the k queries, so the derivative in β_0 gives $k\beta_0 = \sum_i e_i - \beta^\top \sum_i \tilde{x}_i = k\bar{e}$. Then β minimizes $\|(e - \bar{e}\mathbf{1}) - \tilde{X}\beta\|^2 + \lambda\|\beta\|^2$, with the stated unique minimizer, and the eigenvalues of $(\tilde{X}^\top \tilde{X} + \lambda I)^{-1}$ are at most $1/\lambda$ since $\tilde{X}^\top \tilde{X}$ is positive semidefinite. □

The speed factor is one end of the ridge correction's family, and leave-one-out picks the point the k queries support, noisily when k is as small as 5. Template offsets put all their freedom in the templates instead.

Choosing the k queries. *Random* follows the arrival order. *Round robin* takes one query of each template in turn, in random order; *cheapest per template* is round robin from each template's fastest source query. *k-center* adds the query farthest from those chosen in the standardized feature space (Gonzalez 1985; Sener and Savarese 2018), *D-optimal* the one that most increases $\det(I + X^\top X)$ (Wynn 1970; Mitchell 1974), and *plan class* alternates parallel and serial plans. Each rule derives its r-th sequence from the r-th order, so comparisons are paired. Algorithm 1 combines the pieces.

5 Experimental Setup

Environments (Table 1). M2, M4 and M6 pin every PostgreSQL process to one or two virtual processors, which a plan's leader and parallel workers (at most two per Gather node) then share; the proposal limited all of WSL instead, but pinning changes only the database's processors. All run PostgreSQL 18.6 from one build, without JIT, with one set of settings apart from M5's buffer pool, on copies of M1's database files, so every saved plan, with its estimates, is identical in all six. The data fit in memory everywhere. The proposal planned M1 to M3 and TPC-DS on M1 (the course study); the extended study added the rest.

Workloads. TPC-H at scale factor 1 (TPC 2022): 680 queries from 22 templates, the 420 of an earlier pilot plus 20 with widened ranges for each of the 13 templates with range parameters, so that runtime varies inside a template. TPC-DS at scale factor 1 (TPC 2024): up to 5 parameter draws per template, with templates over 30 s in a screening run on M1 dropped before any label existed, which leaves 482 queries from 93 templates. Templates whose first queries share at least 85% of their tokens are near copies (37 and 82 differ only in the sales channel) and form 75 families. The pools hold 476 TPC-H (70%) and 294 TPC-DS queries (61%, 3 of 5 in most templates).

Table 1: The six environments. Pass: minutes for one pass over a workload (median of three). Noise: Q-error between two passes over the same query, median / 90th percentile.

		TPC-H		TPC-DS	
	Setup	Pass	Noise	Pass	Noise
M1	Laptop, Core Ultra 7 155H (22 threads), WSL2, 16 GB	21	1.07 / 1.23	28	1.09 / 1.60
M2	M1, PostgreSQL pinned to one virtual processor	51	1.07 / 1.25	38	1.08 / 1.21
M3	Server VM, 64 vCPUs of an AMD EPYC 9754, 1 TB	34	1.05 / 1.12	41	1.05 / 1.20
M4	M3, PostgreSQL pinned to one virtual processor	75	1.07 / 1.19	61	1.03 / 1.10
M5	M1 with a 128 MB buffer pool instead of 2 GB	25	1.03 / 1.12	34	1.05 / 1.25
M6	M3, PostgreSQL pinned to two virtual processors	44	1.08 / 1.20	46	1.04 / 1.15

Labels. In each pass a query runs once to warm the cache and three times with `EXPLAIN (ANALYZE, TIMING OFF)`; the label is the median over three passes, in random orders, of the median of the three runs. A pass whose level shifts by over 10%, or during which the laptop slept or ran on battery, is repeated.

Protocol. C1 compares the source models (70/30 split inside every template, 5 seeds, and one template held out at a time). C2 adapts with $k = 5$ to 200 target queries over 20 orders; its harder variant holds a template out of the source model and the $k = 25$ queries (5 orders). C3 continues learning, and C4 holds out one TPC-DS family at a time on M1, with every setting frozen on TPC-H. The proposal fixed one primary test per question, everything else being descriptive: for Q1, the ridge correction against the speed factor at $k = 25$ from M1 to M3 on TPC-H; for Q2, the correction model against the cost line in C4, where I replaced single templates by families before any TPC-DS label existed, so that no near copy of a held-out template stays in training. A test succeeds if the 95% interval of the ratio lies below 1. After the course study I designed the template methods on the TPC-H pairs among M1 to M3 and committed them with five hypotheses before any label of 18 new pairs existed: the 12 TPC-DS pairs among M1 to M4 and the 6 TPC-H pairs with M4. All evaluations ran on one Linux server.

6 Results

6.1 How runtimes shift

Across the 30 ordered pairs of each workload, the template share (1) of the log slowdown is 0.69 to 0.94 (0.68 to 0.93 corrected for the number of templates), against at most 0.08 (TPC-H) and 0.22 (TPC-DS) between two passes; for TPC-H from M1 to M2 and M3 it is 0.86 and 0.80, far above the 0.14 between the pilot's unchanged passes that the proposal set as its threshold. What separates templates depends on the change (Figure 1a). On one processor, parallel plans slow by 2.66 times and serial ones by 1.12, and that one bit explains 0.61 of the variance; on two processors (M3 to M6) parallel plans slow by only 1.32. From laptop to server, both kinds slow alike (2.07 and 1.81) but short queries more than long ones, and log source runtime explains 0.52 (TPC-H) and 0.64 (TPC-DS) of the variance. The small buffer pool moves the median by only 1.08 yet slows TPC-H's serial plans by 1.47, though parallelism and source runtime together explain only 0.08 of the variance. Per-node timings show operators inside the parallel part of a plan taking 2.0 to 3.2 times as long on one processor, while serial operators keep their speed. A ridge model of the slowdown on the 19 features, fitted without the template it predicts, explains at most 0.53 of its variance (TPC-DS: 0.38): the template carries much of the shift.

6.2 C1: source models

On seen templates the random forest and the correction model reach a median Q-error of 1.04 to 1.06 (Table 2), the size of the noise between passes. On a held-out template every learned model does much worse (1.4 to 1.9), and only ridge on M2 beats the cost line with an interval below 1, barely (0.83, interval up to 0.998). The correction model, whose lead on unseen templates in the pilot motivated Q2, has a ratio of 0.95 to 1.03 here; on seen templates, which adaptation needs, it is as good as any model.

Table 2: C1 on each machine's TPC-H pool. Seen: 70/30 split in every template, mean over 5 splits (s.d. at most 0.07). Unseen: one template held out at a time; ratio to the cost line over held-out templates, bold if its 95% interval lies below 1. Last row: template mean (seen), global mean (unseen).

	Seen: median Q			Unseen: median Q			Unseen: ratio		
Model	M1	M2	M3	M1	M2	M3	M1	M2	M3
Cost line	1.78	1.78	1.48	1.84	1.98	1.53	–	–	–
Ridge	1.20	1.20	1.17	1.64	1.47	1.51	0.91	**0.83**	0.97
Random forest	1.06	1.06	1.04	1.57	1.67	1.42	0.93	0.89	0.97
Correction model	1.05	1.06	1.04	1.60	1.92	1.45	0.95	1.03	1.01
Template / global mean	1.16	1.21	1.13	2.07	2.25	1.81	1.17	1.22	1.26

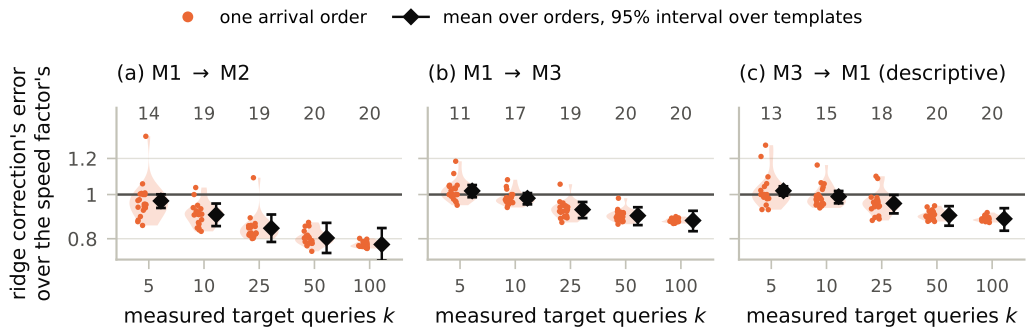


Figure 2: Q1's comparison, order by order: the ridge correction's error over the speed factor's in each of the 20 arrival orders (dots, violins), and the mean with its 95% interval over templates (diamonds). Above each violin: orders in which the ridge correction is better.

6.3 Q1: adapting with k measured queries

The primary test. Without adaptation, M1's model errs by 1.93 on M3 (Figure 1b). A speed factor from $k = 5$ queries removes most of this (1.26, a share 0.69 of the gap) and then barely improves (1.24 at $k = 100$), since one factor cannot follow template slowdowns from 1.25 to 3.10 times. The ridge correction keeps improving: at $k = 25$ from M1 to M3 its error is 0.93 times the speed factor's (95% interval 0.89 to 0.96), lower on 16 of 22 templates, so the primary test succeeds. Over arrival orders (Figure 2) it wins in only 11 to 14 of 20 at $k = 5$: there, from M1 to M3, leave-one-out chose the smallest penalty (an exact fit of the five errors) in 5 orders and the largest (the speed factor, Proposition 1) in 5. From $k = 50$ it wins in all. From M1 to M2, where one processor slows parallel plans far more than serial ones, the speed factor stays at 1.43 even at $k = 100$, and the ridge correction, which sees the planned workers, reaches 0.90 times the speed factor's error at $k = 10$ (0.85 to 0.96) and 0.78 at $k = 100$.

Template offsets. The ridge correction follows the shift only as far as the features describe it (Section 6.1). Table 3 compares all methods over the 30 pairs of each workload at the budget where each template can have one measured query. With round-robin selection, template offsets reach 1.12 on TPC-H and 1.19 on TPC-DS and the template correction 1.10 and 1.19, against 1.08 and 1.18 for the same correction on a model trained on the target itself: once every template has a query, the move costs little. At $k = 25$ the template correction reaches 1.08 (M1 to M3) and 1.10 (M1 to M2) against 1.16 and 1.21 for the ridge correction with random selection (ratios 0.93 [0.90, 0.95] and 0.91 [0.86, 0.95]) and closes a share 0.95 of the gap from M1 to M3, where the ridge correction needs $k = 100$ to close 0.9. Its interval against the ridge correction lies below 1 in 30 of 30 TPC-H ($k = 25$) and 30 of 30 TPC-DS pairs ($k = 100$). 1-NN on the same queries does as well, and a forest on the errors beats the ridge correction on TPC-DS (1.28). A lookup without a model does worse (1.24 on TPC-H), and the two refits lose even to the template correction with random selection (1.17).

Table 3: Error at the budget that allows one query per template, geometric mean over the 30 ordered pairs of a workload. Last row: the template correction on a model trained on the target itself (no move), over the six environments.

Method	Selection	TPC-H, $k = 25$	TPC-DS, $k = 100$
No adaptation		1.83	1.96
Speed factor	random	1.37	1.45
Ridge correction	random	1.24	1.38
Target only	random	1.51	1.80
Pooled refit	random	1.36	1.36
Forest on the errors	random	1.22	1.28
Affine+parallel	random	1.27	1.36
Template lookup (no model)	round robin	1.24	1.24
Template offsets	round robin	1.12	1.19
1-NN	round robin	1.10	1.19
Template correction	round robin	1.10	1.19
Oracle (whole target pool)		1.05	1.26
Template correction, no move	round robin	1.08	1.18

TPC-DS and small budgets. By intervals, on TPC-DS the ridge correction is worse than the speed factor in 30 of 30 pairs at $k = 5$ and 24 at $k = 10$, and better in 2, 14 and 20 at $k = 25$, 50 and 100 (Figure 1c); on TPC-H it is better in 26 at $k = 25$. With at most 4 parameters, affine+parallel is safer before every template has a query: at $k = 10$ it beats the speed factor in 20 TPC-H pairs and loses in 0, and wins in 10 and loses in 7 on TPC-DS.

6.4 Which queries to measure

Covering the templates matters about as much as the correction: on TPC-H at $k = 25$, round robin takes the ridge correction from 1.24 to 1.17 and the template correction from 1.17 to 1.10. Without template labels, k-center selection with 1-NN comes close on TPC-H (1.13) and matches the template correction on TPC-DS at $k = 100$ (1.18 against 1.19), since queries of one template have near plans. D-optimal selection picks long TPC-DS queries (187.7 s of runtime against 109.9 s for round robin), and the cheapest query per template saves time but costs accuracy (1.17 on TPC-H), since short queries shift differently. If each of the k queries is timed once, even by its warm-up run, instead of by its three-pass label, the template correction still reaches 1.14 (TPC-H) and 1.25 (TPC-DS), and the no-move control rises about as much (1.12 and 1.24).

6.5 C3: continued learning

Over the whole pool, refitting the ridge correction on all target queries reaches 1.08 (M1 to M3) and 1.08 (M1 to M2), against 1.12 and 1.16 for the last 50. In a stream that switches from M1 to M2 without warning, scored test-then-train over 5 orders, the error jumps from 1.05 to 2.30; refitting on the last 50 recovers within 54 queries and ends at 1.12, refitting on all queries at 1.69, and never updating at 2.45. Over the extended study's 60 streams the template correction on the last 50 ends at 1.15, the ridge correction at 1.21 (last 50) and 1.49 (all), and no updating at 1.91.

6.6 Q2: unseen templates

On seen templates the ridge correction lowers most errors (Figure 3a), but on a template held out of the source model and the $k = 25$ queries it gains nothing over the speed factor (b): the ratio is 1.03 (0.96 to 1.10) from M1 to M3 and 1.003 (0.87 to 1.16) from M1 to M2, and over all pairs of the extended study its interval is never below 1 and is above 1 in 1 TPC-H and 6 TPC-DS pairs. Retraining helps little: the speed factor is within 1.12 times the error of the oracle retrained without the template in every pair, so what remains is the model's error on a new template.

Q2's primary test fails (Figure 3c): with one TPC-DS family held out at a time, the correction model's error is 1.04 times the cost line's (95% interval 0.90 to 1.19), better on 36 of 75 families, and both do poorly (median Q-errors 2.15 and 2.13, 90th percentiles 11.76 and 11.15). Holding

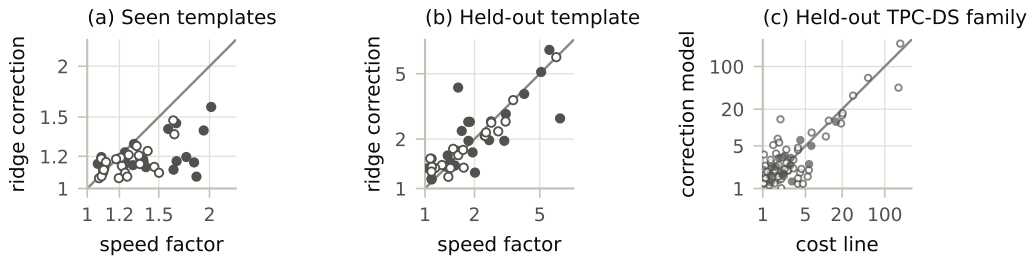


Figure 3: Median Q-error per TPC-H template at $k = 25$ (a, b; filled: M1→M2, hollow: M1→M3) and per TPC-DS family in C4 (c; filled: families of several templates); below the diagonal the learned correction wins. (a) Seen templates; (b) a template held out of the source model and the k queries; (c) a held-out family.

out single templates flatters the correction model, 0.93 (0.82 to 1.06), because a near copy of the held-out template stays in training; even so the interval contains 1.

6.7 Confirmatory tests and other source models

The five hypotheses fixed before the 18 new pairs existed hold under their registered rules: (H1) the template share exceeds 0.5 in every pair (smallest 0.74); (H2) at $k = 100$ the template correction with round robin beats the speed factor in 18 of 18 pairs and the ridge correction with random selection in 16; (H3) at $k = 10$ affine+parallel beats the speed factor in 10 of 12 pairs between one processor and several, meeting the registered three quarters (8 by intervals); (H4) on new templates the speed factor is within 1.15 times the oracle's error (largest 1.08); (H5) at $k = 100$ k-center with 1-NN beats random selection with the ridge correction in 18 of 18 pairs (16 by intervals). With a random forest, boosted trees, ridge regression or the cost line as f_s, the template correction beats the speed factor in all 240 combinations of source model, workload and pair.

7 Discussion and Limitations

For a database that moves to a new machine, the results suggest a recipe: measure one query of each template that matters, chosen round robin, and add to the old model a per-template offset on top of a line that treats parallel plans separately. In the terms of Section 3, the shift δ is mostly a per-template effect: one measured query estimates a template's offset, and the source model supplies the variation inside the template. This came within about 2% of the same correction on a model trained on the new machine, for about 23.6 s of TPC-H runtime. The ridge correction proposed for this course beats a speed factor on TPC-H after 10 to 25 queries but on TPC-DS only from about 50, because the features describe only part of the shift.

The study uses one database system at scale factor 1 (data in memory), queries under 30 s run warm and one at a time, and two physical x86 machines, both virtualized; the 30 pairs of a workload reuse six environments' labels, so they are not independent replications. Plans were held fixed, while a real move may change them, and pinning keeps two workers per Gather node. Errors are per-template medians, and only C4 reports tails. I designed the template methods after seeing the TPC-H pairs among M1 to M3, which is why they were confirmed on pairs measured later.

8 Conclusion

A runtime model moved to a new machine needs no new training set: on TPC-H a ridge correction from 25 measured queries beats a speed factor (Q1), and one query per template with a per-template offset comes close to a model trained there. On a template without measured queries no correction reliably beats the speed factor, and the cost-anchored model is no better than the cost line (Q2). Next steps are other database systems, larger data and plans that change with the machine.

Acknowledgement

References

Akdere, M., U. Çetintemel, M. Riondato, E. Upfal, and S. B. Zdonik (2012). "Learning-based Query Performance Modeling and Prediction". In: *Proceedings of the 28th IEEE International Conference on Data Engineering (ICDE)*, pp. 390–401.

Breiman, L. (2001). "Random Forests". *Machine Learning*, vol. 45, no. 1, pp. 5–32.

Didona, D., F. Quaglia, P. Romano, and E. Torre (2015). "Enhancing Performance Prediction Robustness by Combining Analytical Modeling and Machine Learning". In: *Proceedings of the 6th ACM/SPEC International Conference on Performance Engineering (ICPE)*, pp. 145–156.

Efron, B. and R. J. Tibshirani (1993). "An Introduction to the Bootstrap". New York: Chapman & Hall.

Gama, J., R. Sebastião, and P. P. Rodrigues (2013). "On Evaluating Stream Learning Algorithms". *Machine Learning*, vol. 90, no. 3, pp. 317–346.

Ganapathi, A., H. A. Kuno, U. Dayal, J. L. Wiener, A. Fox, M. I. Jordan, and D. A. Patterson (2009). "Predicting Multiple Metrics for Queries: Better Decisions Enabled by Machine Learning". In: *Proceedings of the 25th IEEE International Conference on Data Engineering (ICDE)*, pp. 592–603.

Golub, G. H., M. Heath, and G. Wahba (1979). "Generalized Cross-Validation as a Method for Choosing a Good Ridge Parameter". *Technometrics*, vol. 21, no. 2, pp. 215–223.

Gonzalez, T. F. (1985). "Clustering to Minimize the Maximum Intercluster Distance". *Theoretical Computer Science*, vol. 38, pp. 293–306.

Heinrich, R., M. Luthra, J. Wehrstein, H. Kornmayer, and C. Binnig (2025). "How Good are Learned Cost Models, Really? Insights from Query Optimization Tasks". *Proceedings of the ACM on Management of Data*, vol. 3, no. 3, Article 172, pp. 1–27.

Hilprecht, B. and C. Binnig (2022). "Zero-Shot Cost Models for Out-of-the-box Learned Cost Prediction". *Proceedings of the VLDB Endowment*, vol. 15, no. 11, pp. 2361–2374.

Hoerl, A. E. and R. W. Kennard (1970). "Ridge Regression: Biased Estimation for Nonorthogonal Problems". *Technometrics*, vol. 12, no. 1, pp. 55–67.

Jamshidi, P., N. Siegmund, M. Velez, C. Kästner, A. Patel, and Y. Agarwal (2017). "Transfer Learning for Performance Modeling of Configurable Systems: An Exploratory Analysis". In: *Proceedings of the 32nd IEEE/ACM International Conference on Automated Software Engineering (ASE)*, pp. 497–508.

Jamshidi, P., M. Velez, C. Kästner, and N. Siegmund (2018). "Learning to Sample: Exploiting Similarities Across Environments to Learn Performance Models for Configurable Systems". In: *Proceedings of the 26th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*, pp. 71–82.

Kurmanji, M. and P. Triantafillou (2023). "Detect, Distill and Update: Learned DB Systems Facing Out of Distribution Data". *Proceedings of the ACM on Management of Data*, vol. 1, no. 1, Article 33, pp. 1–27.

Kuzborskij, I. and F. Orabona (2013). "Stability and Hypothesis Transfer Learning". In: *Proceedings of the 30th International Conference on Machine Learning (ICML)*. Vol. 28. Proceedings of Machine Learning Research, pp. 942–950.

Leis, V., A. Gubichev, A. Mirchev, P. Boncz, A. Kemper, and T. Neumann (2015). "How Good Are Query Optimizers, Really?" *Proceedings of the VLDB Endowment*, vol. 9, no. 3, pp. 204–215.

Liang, Z., X. Chen, Y. Xia, R. Ye, H. Chen, J. Xie, and K. Zheng (2024). "DACE: A Database-Agnostic Cost Estimator". In: *Proceedings of the 40th IEEE International Conference on Data Engineering (ICDE)*, pp. 4925–4937.

Lu, B., J. Yang, W. Jiang, Y. Shi, and S. Ren (2021). "One Proxy Device Is Enough for Hardware-Aware Neural Architecture Search". *Proceedings of the ACM on Measurement and Analysis of Computing Systems*, vol. 5, no. 3, pp. 1–34.

Marcus, R. and O. Papaemmanouil (2019). "Plan-Structured Deep Neural Network Models for Query Performance Prediction". *Proceedings of the VLDB Endowment*, vol. 12, no. 11, pp. 1733–1746.

Mitchell, T. J. (1974). "An Algorithm for the Construction of D-Optimal Experimental Designs". *Technometrics*, vol. 16, no. 2, pp. 203–210.

Moerkotte, G., T. Neumann, and G. Steidl (2009). "Preventing Bad Plans by Bounding the Impact of Cardinality Estimation Errors". *Proceedings of the VLDB Endowment*, vol. 2, no. 1, pp. 982–993.

Pedregosa, F. et al. (2011). "Scikit-learn: Machine Learning in Python". *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830.

PostgreSQL Global Development Group (2026). “PostgreSQL Documentation: EXPLAIN”. Accessed September 2026.

Rieger, M. and T. Neumann (2025). “T3: Accurate and Fast Performance Prediction for Relational Database Systems With Compiled Decision Trees”. *Proceedings of the ACM on Management of Data*, vol. 3, no. 3, pp. 1–27.

Sener, O. and S. Savarese (2018). “Active Learning for Convolutional Neural Networks: A Core-Set Approach”. In: *Proceedings of the 6th International Conference on Learning Representations (ICLR)*.

Shimodaira, H. (2000). “Improving Predictive Inference under Covariate Shift by Weighting the Log-Likelihood Function”. *Journal of Statistical Planning and Inference*, vol. 90, no. 2, pp. 227–244.

Steyerberg, E. W., G. J. J. M. Borsboom, H. C. van Houwelingen, M. J. C. Eijkemans, and J. D. F. Habbema (2004). “Validation and Updating of Predictive Logistic Regression Models: A Study on Sample Size and Shrinkage”. *Statistics in Medicine*, vol. 23, no. 16, pp. 2567–2586.

Sun, J. and G. Li (2019). “An End-to-End Learning-based Cost Estimator”. *Proceedings of the VLDB Endowment*, vol. 13, no. 3, pp. 307–319.

Transaction Processing Performance Council (2022). “TPC Benchmark H (Decision Support) Standard Specification, Revision 3.0.1”. Data and queries from TPC-H tools 2.17.3 (gregrahn/tpch-kit, commit 852ad0a); later revisions changed only reporting and pricing rules.

Transaction Processing Performance Council (2024). “TPC Benchmark DS Standard Specification, Version 4.0.0”. Data and queries from TPC-DS tools 2.10.0 (gregrahn/tpcds-kit, commit 5a3a817).

Valov, P., J. Guo, and K. Czarnecki (2020). “Transferring Pareto Frontiers across Heterogeneous Hardware Environments”. In: *Proceedings of the ACM/SPEC International Conference on Performance Engineering (ICPE)*, pp. 12–23.

Valov, P., J.-C. Petkovich, J. Guo, S. Fischmeister, and K. Czarnecki (2017). “Transferring Performance Prediction Models Across Different Hardware Platforms”. In: *Proceedings of the 8th ACM/SPEC International Conference on Performance Engineering (ICPE)*, pp. 39–50.

Venkataraman, S., Z. Yang, M. Franklin, B. Recht, and I. Stoica (2016). “Ernest: Efficient Performance Prediction for Large-Scale Advanced Analytics”. In: *Proceedings of the 13th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, pp. 363–378.

Vergouwe, Y., D. Nieboer, R. Oostenbrink, T. P. A. Debray, G. D. Murray, M. W. Kattan, H. Koffijberg, K. G. M. Moons, and E. W. Steyerberg (2017). “A Closed Testing Procedure to Select an Appropriate Method for Updating Prediction Models”. *Statistics in Medicine*, vol. 36, no. 28, pp. 4529–4539.

Wu, P. and Z. G. Ives (2024). “Modeling Shifting Workloads for Learned Database Systems”. *Proceedings of the ACM on Management of Data*, vol. 2, no. 1, Article 38, pp. 1–27.

Wu, W., Y. Chi, S. Zhu, J. Tatemura, H. Hacıgümüş, and J. F. Naughton (2013). “Predicting Query Execution Time: Are Optimizer Cost Models Really Unusable?” In: *Proceedings of the 29th IEEE International Conference on Data Engineering (ICDE)*, pp. 1081–1092.

Wu, Z. et al. (2024). “Stage: Query Execution Time Prediction in Amazon Redshift”. In: *Companion of the 2024 International Conference on Management of Data (SIGMOD-Companion)*, pp. 280–294.

Wynn, H. P. (1970). “The Sequential Generation of D-Optimum Experimental Designs”. *The Annals of Mathematical Statistics*, vol. 41, no. 5, pp. 1655–1664.

Yan, Y., H. Wang, J. Huang, D. Zhong, T. Yu, K. Zhang, M. Yang, and T. Wang (2024). “QCFE: An Efficient Feature Engineering for Query Cost Estimation”. In: *Proceedings of the 40th IEEE International Conference on Data Engineering (ICDE)*, pp. 4302–4315.

Yang, J., S. Wu, D. Zhang, J. Dai, F. Li, and G. Chen (2023). “Rethinking Learned Cost Models: Why Start from Scratch?” *Proceedings of the ACM on Management of Data*, vol. 1, no. 4, Article 255, pp. 1–27.

Yu, W. and S. Luo (2023). “Less is More: Towards Lightweight Cost Estimator for Database Systems”. *arXiv preprint arXiv:2303.10983*.

Zhao, Y., G. Cong, J. Shi, and C. Miao (2022). “QueryFormer: A Tree Transformer Model for Query Plan Representation”. *Proceedings of the VLDB Endowment*, vol. 15, no. 8, pp. 1658–1670.

Zhu, Q. and P.-Å. Larson (1994). “A Query Sampling Method for Estimating Local Cost Parameters in a Multidatabase System”. In: *Proceedings of the 10th International Conference on Data Engineering (ICDE)*, pp. 144–153.